Reducing Bus Bunching on a Loop Route with Q-Learning Holding Control

Faiz Imdad (B23CM1016)
GitHub: `https://github.com/Faizimdad/bus-bunching-rl`

1 Problem

On frequent bus routes, buses that start evenly spaced drift into *bunches*. A slightly late bus finds more waiting passengers, dwells longer and falls further behind, while the bus behind it finds fewer passengers and catches up. Passengers then wait for a long gap followed by several buses arriving together. Agencies counter this by *holding* buses at stops, but holding also delays the people on board. We cast the holding decision as a reinforcement-learning problem: at each stop, an agent decides whether to hold a bus for 0, 2 or 4 minutes.

2 Environment

We built a time-stepped simulator ($\Delta t = 0.1$ min) from scratch. A loop route has N=12 stops served by M=4 buses (capacity 70). Segment run times are log-normal (mean 2.5 min, CV 0.25), resampled every departure. Passengers arrive at each stop as a Poisson process ($\lambda = 0.6$ pax/min); each on-board passenger alights at a stop with probability 0.15. Dwell time is $d = 0.25 + 0.08\, n_{\text{board}} + 0.04\, n_{\text{alight}}$ min, and passengers who arrive while the bus is dwelling or held also board. This dwell feedback alone reproduces bunching: with no control, 46% of arrivals at stops follow the previous bus by less than a quarter of the mean headway (Fig. 1, left). Episodes last 240 min; the first 30 min (warm-up) are excluded from all metrics. Initial positions are perturbed by up to ±15% of the ideal spacing.

3 RL Formulation

Decision epochs. A decision is made whenever a bus has finished boarding and is ready to depart; the interval between decisions is variable, so this is a semi-Markov decision process. All buses share one policy (parameter sharing), giving ≈1,100 decisions per episode.

State space S. For bus i at a decision we observe g_f, the spatial gap to the bus ahead, and g_b, the gap to the bus behind, both divided by the ideal spacing N/M (so $g = 1$ is perfect), plus the load fraction $\ell = \text{load}/C$. Each gap is discretised into 7 bins with edges $\{0.3, 0.6, 0.85, 1.15, 1.4, 1.7\}$ and the load into 3 bins (edges 0.3, 0.6), giving $|S| = 7 \times 7 \times 3 = 147$ discrete states.

Action space A. Discrete: hold $h \in \{0, 2, 4\}$ min ($|A| = 3$).

Reward R. The cost of an action is realised when the same bus reaches its next stop and observes new gaps g'_f, g'_b:

$$r = -\tfrac{1}{2}\left[(g'_f - 1)^2 + (g'_b - 1)^2\right] \; - \; w_h\, h\,(0.5 + \ell), \quad (1)$$

with gaps clipped at 2.5 and $w_h = 0.05$. The first term rewards even spacing in both directions; the second penalises holding, more so when many passengers are on board (the constant 0.5 represents the slower route cycle that holding causes even for an empty bus).

Algorithm. Tabular Q-learning [2, 1], $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$, where s' is the state at the same bus's next decision. We use $\alpha = 0.1$, $\gamma = 0.9$, and ε-greedy exploration with ε decaying linearly from 1.0 to 0.05 over the first 70% of 3,000 training episodes (each with a fresh random seed). The state is small and interpretable, so a table is sufficient and lets us inspect the policy directly (Fig. 4); a DQN would be the natural next step for continuous states.

Baselines. All use the same action set. (i) *No holding.* (ii) *Forward-headway rule*: if $g_f < 0.8$, hold for the largest option not exceeding the time needed to restore 0.8× the ideal gap. (iii) *Two-way rule*: hold for the largest option not exceeding $\frac{1}{2}(g_b - g_f)$ converted to minutes, which moves the bus toward the midpoint between its neighbours (a discretised form of headway-based control [3]).

4 Results

All controllers are evaluated on the same 100 test seeds, disjoint from training and validation seeds. Table 1 and Fig. 2 summarise them. We measure *total cost per passenger* = average wait at stops + held passenger-minutes per boarding, so a controller cannot look good merely by holding buses for long periods.

Controller	Wait	Hold	Total	CV	Bunch%
No holding	12.16	0.00	12.16	1.175	46.0
Forward rule	8.13	1.43	9.56	0.593	0.0
Two-way rule	5.90	1.23	7.14	0.401	2.2
Q-learning	**4.91**	1.68	**6.59**	**0.147**	**0.0**

Table 1: Mean over 100 test episodes (minutes per passenger; CV = coefficient of variation of headways at stops).

The learned policy cuts total passenger cost by 46% versus no holding and by 8% versus the best rule, and reduces headway CV by 2.7× relative to the two-way rule. It holds slightly more than the rules (higher hold delay) but buys a much larger reduction in waiting. Learning is stable: the greedy policy beats the two-way rule on validation seeds from about episode 1,500 onward and the mean reward plateaus after ∼2,200 episodes (curves in the repository, `results/figures/learning_curve.png`).

What was learned. Fig. 4 shows the greedy action at high load. The agent holds when the gap ahead is small and the gap behind is large (the lower-right region), and does not hold when the bus is itself late (large g_f, small g_b, upper-left). This mirrors the logic of the two-way rule, but the agent learns *how much* to hold for each combination from experience rather than from a fixed formula, and it holds for the full 4 min more readily when the bus behind is far away.

Generalisation and robustness. Without retraining, the policy remains the best controller at $\lambda = 0.5$ and

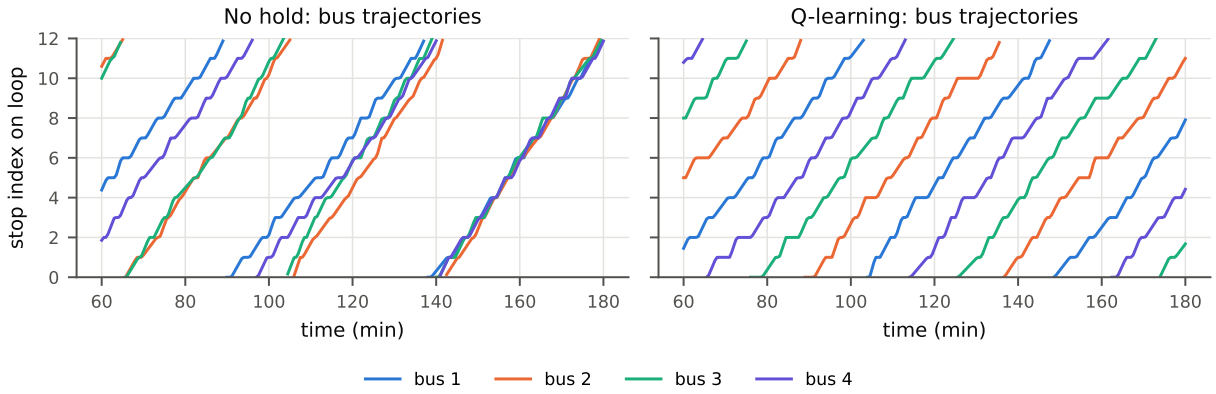


Figure 1: Bus trajectories on the same test episode (each line is one bus; flat steps are dwell/hold). Without control the four buses collapse into one or two platoons; the learned policy keeps them evenly spaced.

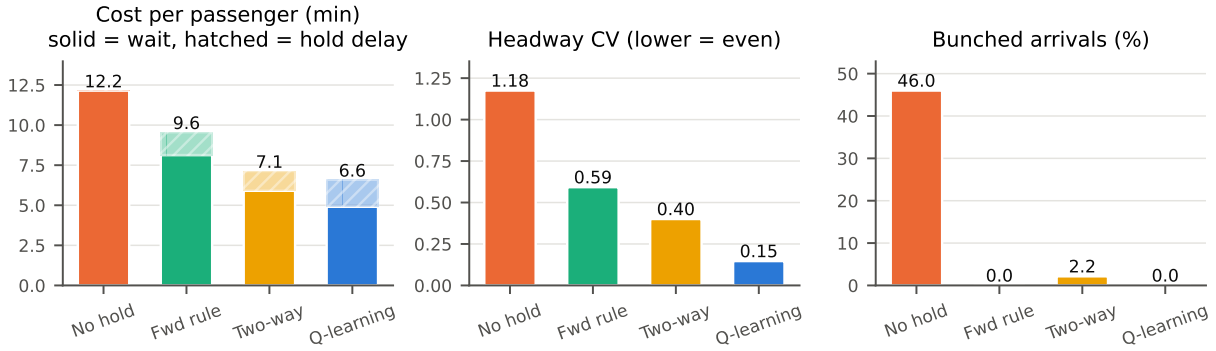


Figure 2: Test results averaged over 100 held-out episodes. Left: passenger cost split into waiting at stops (solid) and delay caused by holding (hatched).

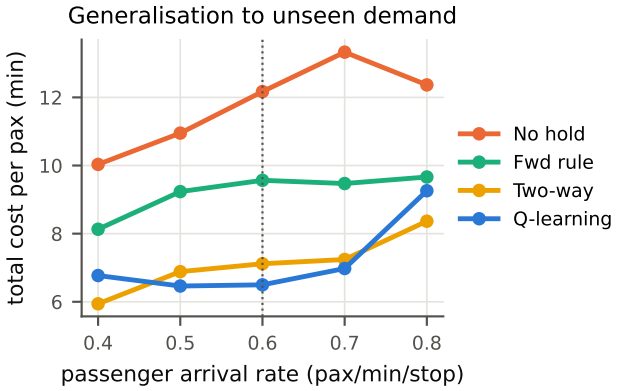


Figure 3: Total cost per passenger as demand varies; the agent was trained only at $\lambda = 0.6$ (dotted line). 40 test seeds per point.

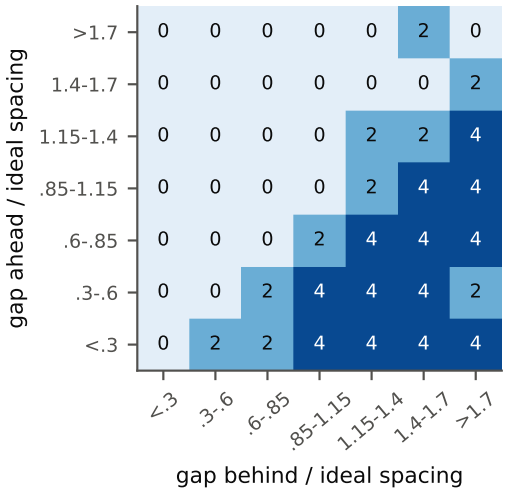


Figure 4: Learned hold (min) at high load ($\ell > 0.6$) as a function of the gap ahead and the gap behind.

0.7 (Fig. 3). At very light demand ($\lambda = 0.4$) the two-way rule is slightly better, because bunching is mild and the agent's holds become unnecessary; at heavy demand ($\lambda = 0.8$) loads fall outside the training distribution and the two-way rule again does better. Training over a range of demand levels would address both cases. Retraining with four further seeds gives total costs of 6.61–6.77 min (headway CV 0.147–0.151; 50 test seeds each), all below the two-way rule, so the result is not a lucky seed. The holding penalty matters: with $w_h = 0$ the agent learns to hold almost always (total cost 26.7 min). Raising w_h from 0.02 to 0.5 trades regularity for less holding (CV 0.148→0.292, total cost 6.61→6.43 min); operators can set w_h to match how they value on-board delay.

5 Conclusion and Limitations

A 147-state Q-learning agent with three actions learns a holding policy that eliminates bunching and lowers total passenger cost below well-known rule-based controllers on a stochastic loop route. Limitations: the simulator is synthetic (not calibrated to a real city), all buses share one policy without communication, and performance degrades under demand far outside training. Natural extensions are training across a range of demand levels, a DQN over continuous gaps, and multi-agent coordination between neighbouring buses.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, 8(3–4):279–292, 1992.

[3] C. F. Daganzo, "A headway-based approach to eliminate bus bunching," *Transportation Research Part B*, 43(10):913–921, 2009.